

SignBridge

Gamified ASL Learning Platform

Project Report

Name: Anaya Azam

Roll no: 2026-S-CYS-12

Section : A

Department: CE - Cybersecurity

Course : CMPE-112L PFDS

Date: 29th June, 2026.



1. Title & Objective

1.1 Title

SignBridge — Gamified ASL (American Sign Language) Learning Platform

1.2 Project Objective

The primary objective of SignBridge is to provide an accessible, engaging, and bilingual platform for learning American Sign Language (ASL). The application bridges the gap between hearing and non-hearing communities by making ASL education interactive, measurable, and culturally inclusive for Pakistani learners through Urdu language support.

Specific objectives include:

- **Educational:** Provide a structured ASL curriculum covering the alphabet (A–Z), common words, and numbers through an intuitive desktop interface.
- **AI Recognition:** Integrate real-time AI-based hand sign recognition using a TensorFlow model and OpenCV camera pipeline, enabling live practice and quiz sessions.
- **Gamification:** Implement a complete gamification system — XP points, level tiers, daily streaks, achievement badges, and a global leaderboard — to sustain learner motivation.
- **Bilingual TTS:** Offer bilingual Text-to-Speech (TTS) output in both English and Urdu using gTTS and deep-translator, making the app inclusive for Urdu-speaking users.
- **UI Modernization:** Replace the original CustomTkinter UI framework with PyQt5 to achieve a more robust, scalable, and professionally styled desktop interface with full QSS stylesheet support.

2. Project Overview

SignBridge is a Python desktop application developed as a final project for CMPE-112L. It functions as a complete self-contained learning management system for ASL, combining computer vision, natural language processing, and database-backed progress tracking within a single standalone application.

The application presents a sidebar-navigated interface with ten distinct screens: Login, Register, Home, Learn, Practice, Quiz, Speak, Stats, Leaderboard, Profile, and Settings. Each screen serves a specific learning or management function.

2.1 Application Architecture Summary

Component	Technology	Role
GUI Framework	PyQt5 5.15+	All UI widgets, layouts, signals & slots
AI / Vision	TensorFlow 2.13+ + OpenCV	Real-time ASL hand sign classification
Database	SQLite + bcrypt	User accounts, progress, quiz history
Charts	Matplotlib (Qt5Agg backend)	Embedded stats visualizations
TTS / Translation	gTTS + deep-translator	English and Urdu speech

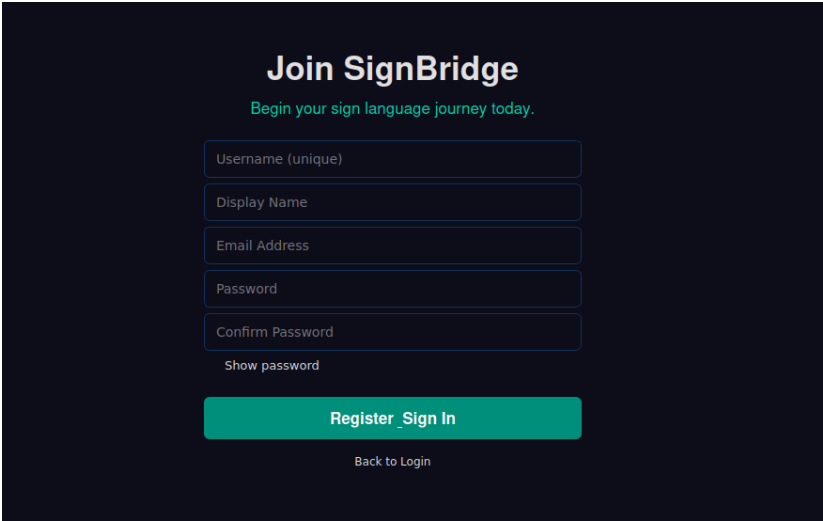
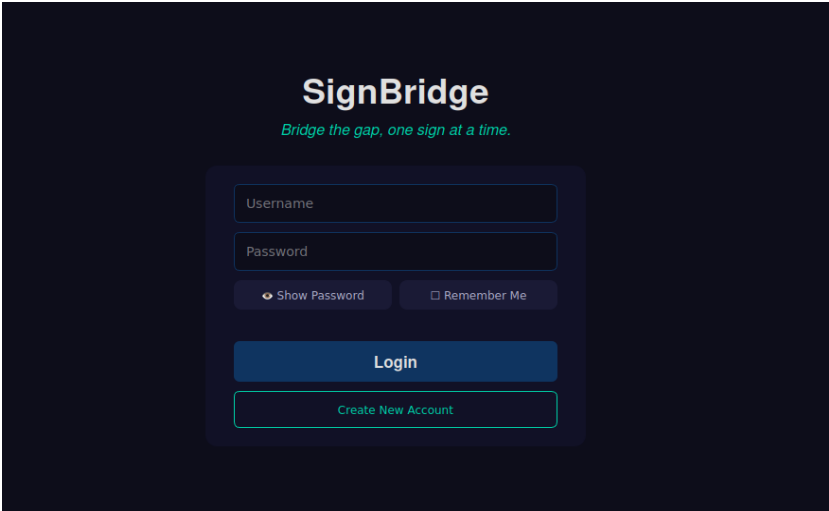
Component	Technology	Role
		synthesis
Audio Playback	pygame + pytsx3	Online and offline audio output
Gesture Detection	MediaPipe (preprocessing)	Landmark extraction for model input

3. Key Aspects

3.1 PyQt5 UI Framework

The most significant technical upgrade from the original prototype is the complete migration from CustomTkinter to PyQt5. PyQt5 is a set of Python bindings for the Qt5 C++ application framework, offering:

- QMainWindow with QStackedWidget for seamless screen transitions without window recreation.
- QSS (Qt Style Sheets) for fine-grained, CSS-like styling across all widgets.
- QTimer for non-blocking camera loops in Practice and Quiz screens.
- FigureCanvasQTAgg for embedding Matplotlib charts natively within PyQt5 layouts.
- QScrollArea and QGridLayout for responsive, scrollable content regions.
- Signal-slot architecture replacing callback-based CustomTkinter event handling.



3.2 Real-Time AI Recognition

The ASLDetector module wraps a TensorFlow SavedModel trained to classify 26 ASL alphabet hand signs. The recognition pipeline operates as follows:

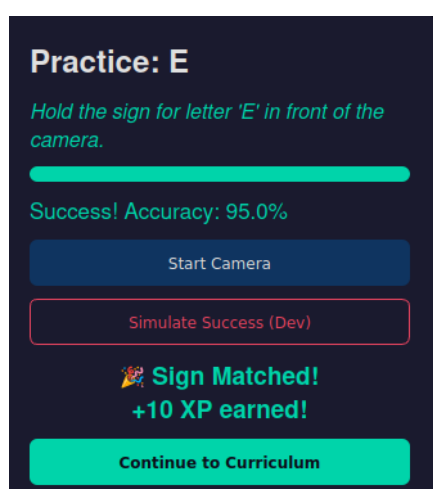
- OpenCV captures frames from the webcam at the configured camera index.
- MediaPipe Hands extracts 21 3D hand landmark coordinates per frame.
- The landmark array is preprocessed and passed to the TensorFlow model for inference.
- A letter buffer (LetterBuffer) smooths predictions over multiple frames, requiring 1.5 seconds of consistent hold for a confirmed match.
- Simulation mode activates when no model file is present, enabling testing via a 'Simulate Success' button.



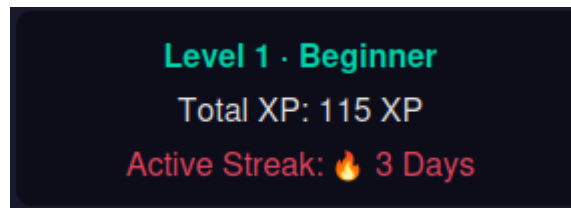
3.3 Gamification Engine

Gamification is central to the SignBridge learning philosophy. The system comprises three independent modules:

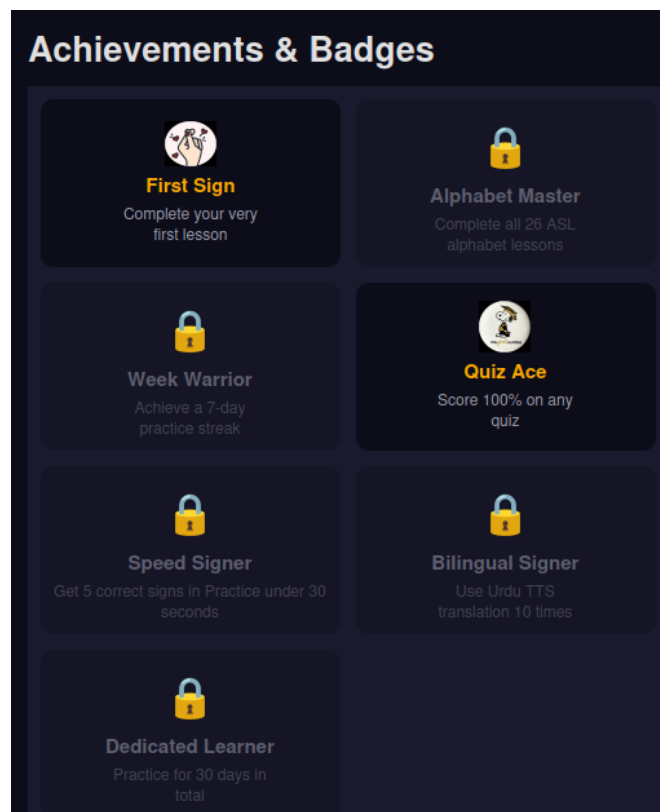
- **XP System:** XP Engine (xp_engine.py): Awards XP for completed lessons, practice sessions, and quiz scores. Tracks cumulative XP and computes the current level tier.



- **Streak System:** Streak Tracker (streak_tracker.py): Records daily login and activity timestamps. A 7-day continuous streak awards a +50 XP bonus. Missing a day resets the streak counter.
-



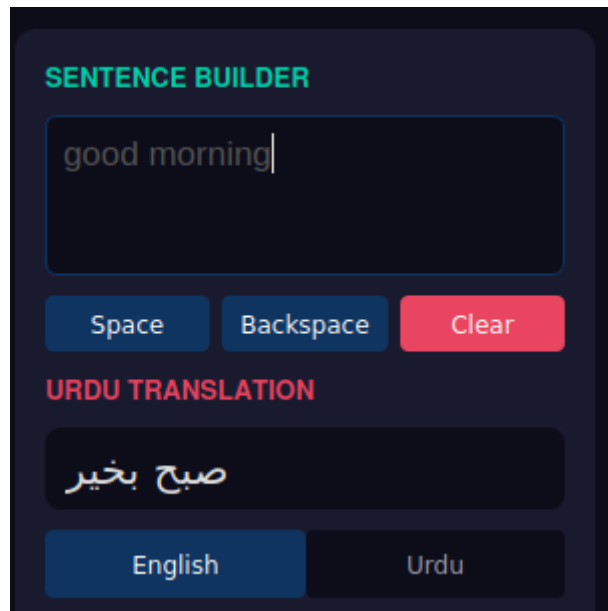
- **Badge System:** Badge Engine (badge_engine.py): Monitors unlock conditions for seven achievement badges — First Sign, Week Warrior, Monthly Master, Alphabetist, Quiz Ace, Bilingual, and Centurion.



3.4 Bilingual Learning Experience

SignBridge targets Pakistani learners specifically by integrating full Urdu-language support:

- The Speak screen accepts ASL letter input and assembles words from signed letters.
- deep-translator (Google Translate backend) converts assembled English words to Urdu.
- gTTS synthesises audio in both English and Urdu with automatic caching to avoid re-generation.
- pyttsx3 provides an offline TTS fallback when the internet is unavailable.



3.5 Spaced Repetition & Quiz Engine

The quiz module uses a simplified spaced repetition system (SRS) to prioritise signs the learner has historically struggled with:

- Quiz sessions present 10 questions drawn from the learner's completed lessons.
- Two question types are supported: FLASHCARD (identify the sign shown) and SIGN_IT (perform the sign via webcam).
- Incorrect answers increase a sign's review priority weight in the spaced_rep module.
- Perfect quiz scores award bonus XP; quiz history is persisted to SQLite.

4. Modular Hierarchy

The project is structured as a Python package with a clear separation of concerns across four major subsystems: authentication, database, UI screens, and core logic modules.

4.1 Top-Level Structure

File / Folder	Type	Purpose
main.py	Entry point	Initialises QApplication, builds window, manages screen routing
settings.json	Config	Persists user preferences (theme, language, camera index, dev mode)
requirements.txt	Config	Python package dependencies
signbridge.db	Database	SQLite database — auto-created on first run
auth/	Package	Login and Registration screens
ui/	Package	All 9 main application screens
modules/	Package	All non-UI business logic sub-packages
database/	Package	DB manager, schema, seed data
assets/	Resources	Sign images, badge icons, avatar PNGs
model/	Resources	TensorFlow SavedModel directory (optional)

4.2 Auth Package

Module	Class	Key Responsibilities
login_screen.py	LoginScreen	Credential entry, bcrypt password verification, remember-me session
register_screen.py	RegisterScreen	New account creation, username uniqueness check, password hashing

4.3 UI Package (PyQt5 Screens)

Module	Class	Description
home.py	HomeScreen	Dashboard: XP bar, streak, recent activity, quick-action buttons
learn.py	LearnScreen	Browsable sign library by unit with reference images and signing tips
practice.py	PracticeScreen	Live webcam feed; hold sign 1.5s to match; XP + badge triggers
quiz.py	QuizScreen	10-question FLASHCARD / SIGN_IT sessions; progress bar; score display
speak.py	SpeakScreen	Sign letters to spell words; Urdu translation; TTS playback
stats.py	StatsScreen	Matplotlib charts: 7-day activity bar, accuracy trend, curriculum donut
leaderboard.py	LeaderboardScreen	Top-25 users by XP with streaks; current user highlighted in teal
profile.py	ProfileScreen	Avatar selector, display name, XP/level/streak summary, badge grid
settings.py	SettingsScreen	Theme, language, camera index, dev mode toggles; save/apply

4.4 Modules Package

4.4.1 Gamification Sub-package

Module	Function
xp_engine.py	award_xp(), get_level_info(), compute_level() — XP award and level tier logic
streak_tracker.py	update_streak(), get_streak(), check_weekly_bonus() — daily streak management
badge_engine.py	check_and_award_badges(), get_user_badges() — condition evaluation and award

4.4.2 Learning Sub-package

Module	Function
lesson_manager.py	get_lessons_by_unit(), mark_lesson_complete(), get_progress() — lesson state
seed_lessons.py	seed_database() — populates DB with all sign lessons on first run

4.4.3 Quiz Sub-package

Module	Function
quiz_engine.py	QuizEngine — generates question sets, scores answers, records results to DB

Module	Function
spaced_rep.py	get_priority_signs(), update_difficulty() — SRS weight management

4.4.4 Recognizer Sub-package

Module	Function
asl_detector.py	ASLDetector — loads TF model, runs inference, returns (label, confidence)
letter_buffer.py	LetterBuffer — smooths predictions over N frames to prevent jitter

4.4.5 TTS Sub-package

Module	Function
tts_engine.py	TTSEngine — generates and caches gTTS audio; pyttsx3 offline fallback; pygame playback
translator.py	translate_to_urdu() — wraps deep-translator Google backend for English-to-Urdu

4.5 Database Package

Module	Purpose
db_manager.py	DatabaseManager — single SQLite connection wrapper; all CRUD operations
schema.sql	Table definitions: users, lessons, user_progress, quiz_results, badges
seed_lessons.py	Called by main.py on startup; idempotent population of lessons table

5. GUI Interface Design

5.1 Design Language & Color System

The entire interface is built around a dark, professional color palette chosen to suggest high-tech precision while remaining readable for extended learning sessions:

Token	Hex Value	Usage
DARK_BG	#0d0d1a	Primary window and content area background
SIDEBAR_BG	#1a1a2e	Navigation sidebar and card backgrounds
ACCENT (Teal)	#00d4aa	Active nav highlight, XP bar, success states, headings
RED_ACCENT	#e94560	Streak indicators, incorrect answers, logout button
BLUE_ACCENT	#0f3460	Hover states, selected elements, secondary cards
TEXT_MAIN	#e0e0e0	Primary body text
TEXT_DIM	#a0a0b0	Secondary labels and metadata

5.2 Layout Structure

The root window uses a QHBoxLayout dividing the screen into two zones:

- Sidebar (QFrame, fixed 200px wide): Contains the SignBridge logo label, eight

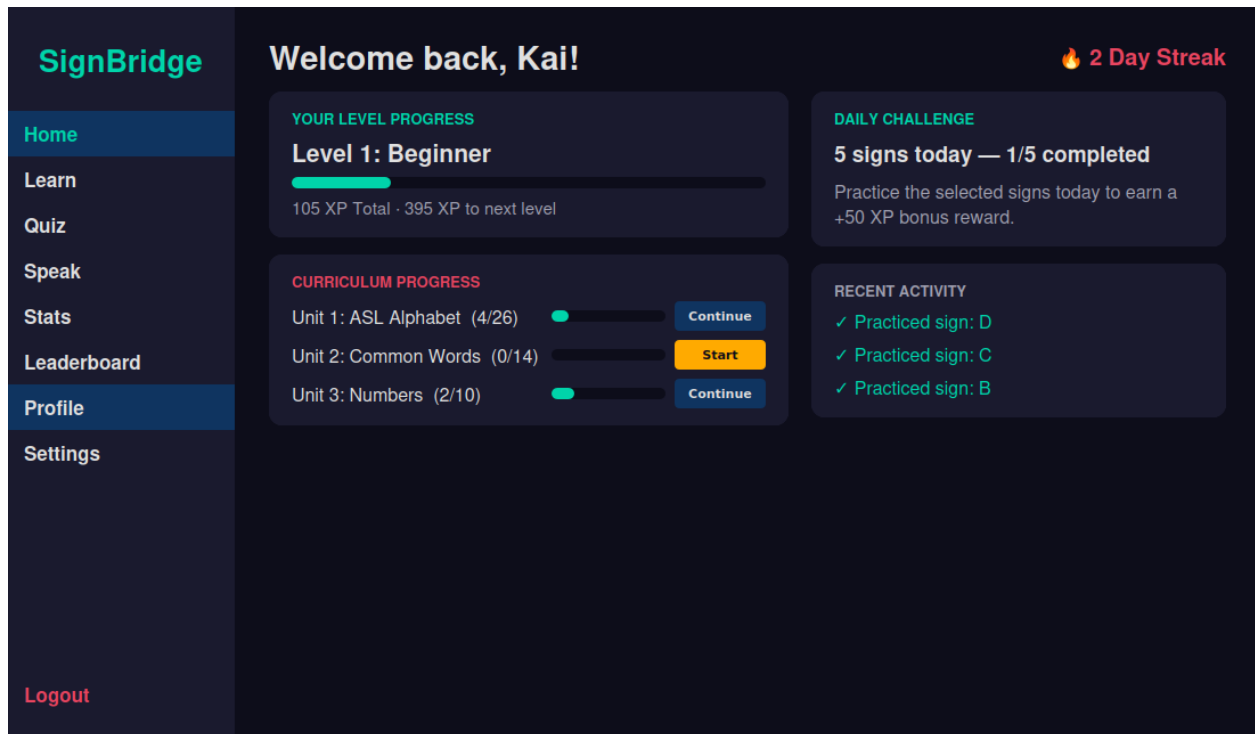
QPushButton navigation items, a stretch spacer, and a Logout button. Active screen is highlighted with BLUE_ACCENT background and TEAL text.

- Content Stack (QStackedWidget): Fills the remaining width. Shows exactly one screen widget at a time. Screen transitions call setCurrentWidget() with no animation, ensuring instant response.

5.3 Screen-by-Screen Design

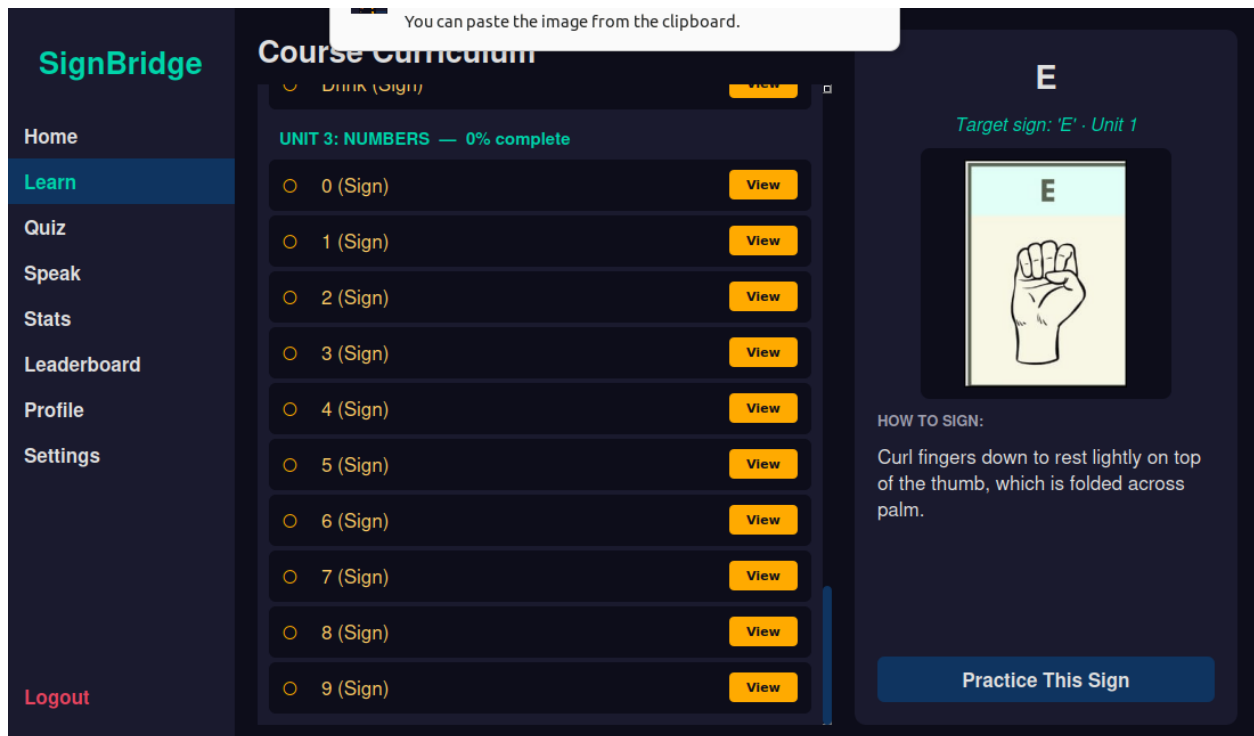
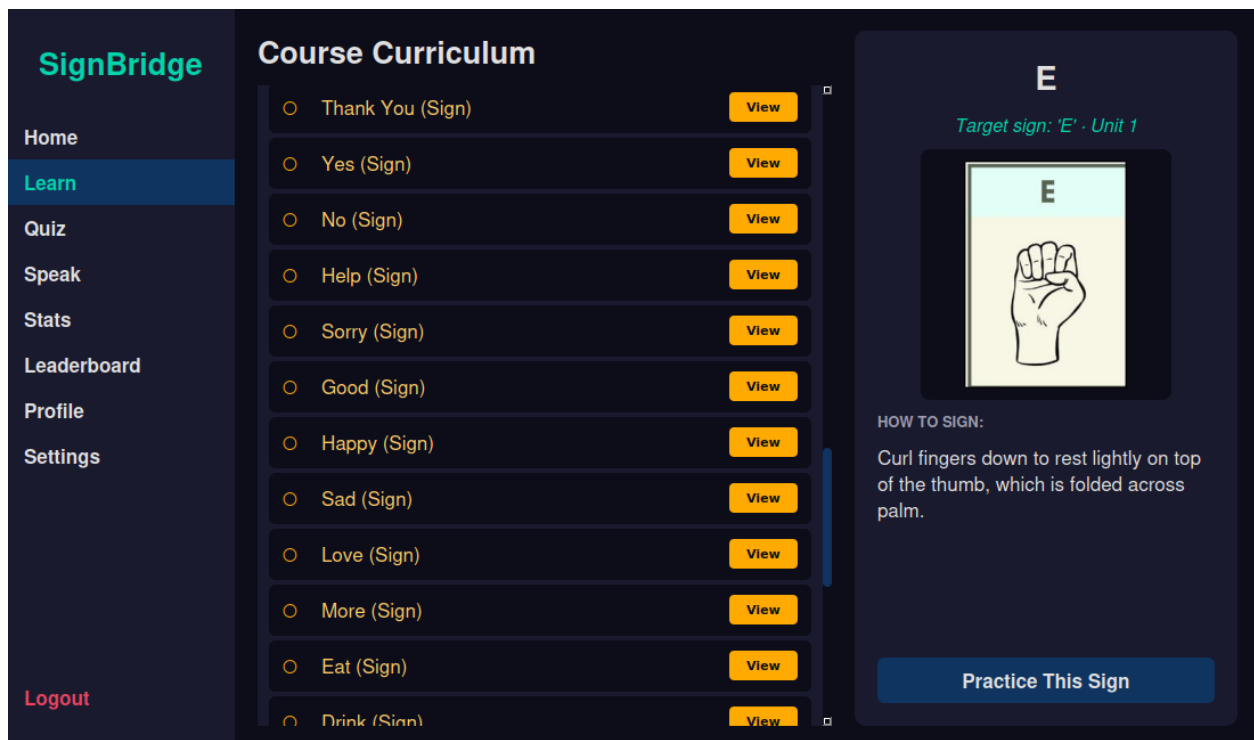
Home Screen

The Home screen is the post-login dashboard. It displays a personalised greeting with the username, a horizontal XP progress bar showing progress toward the next level, and three summary stat cards (Total XP, Current Streak, Signs Learned). Quick-action buttons link directly to Learn, Practice, and Quiz. A recent activity feed shows the last five sign interactions.



Learn Screen

The Learn screen uses a QScrollArea containing a QGridLayout to display all lessons organised by unit. Each lesson tile shows the sign reference image (loaded from assets/signs/), the sign label, a completion checkmark if mastered, and the XP reward. Clicking a tile expands a detail panel with signing tips and a 'Practice This Sign' shortcut.



Practice Screen

The Practice screen is split into a left column (webcam feed rendered via QLabel with QImage) and a right column (current target sign image, confidence bar, and status message). A QTimer fires at 30ms intervals to pull frames from OpenCV and trigger inference. When the detector holds a match for 45 consecutive frames (1.5 seconds), XP is awarded and the next sign queues automatically.

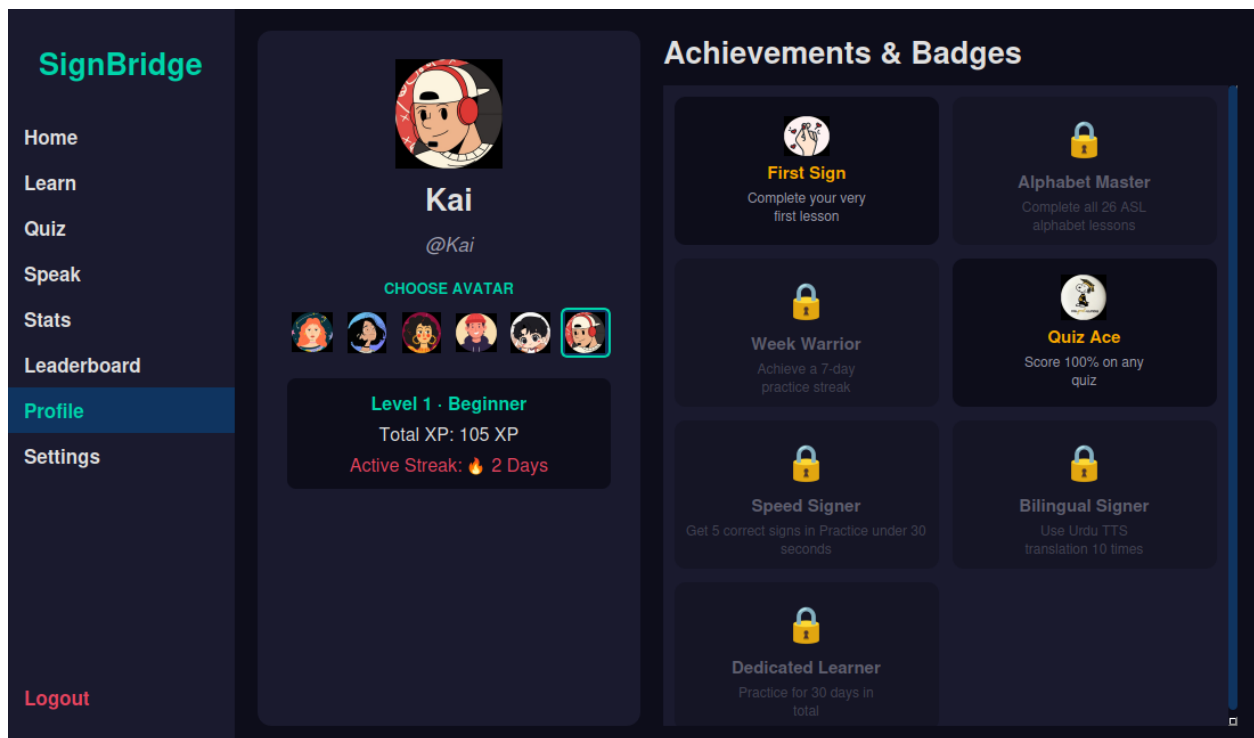
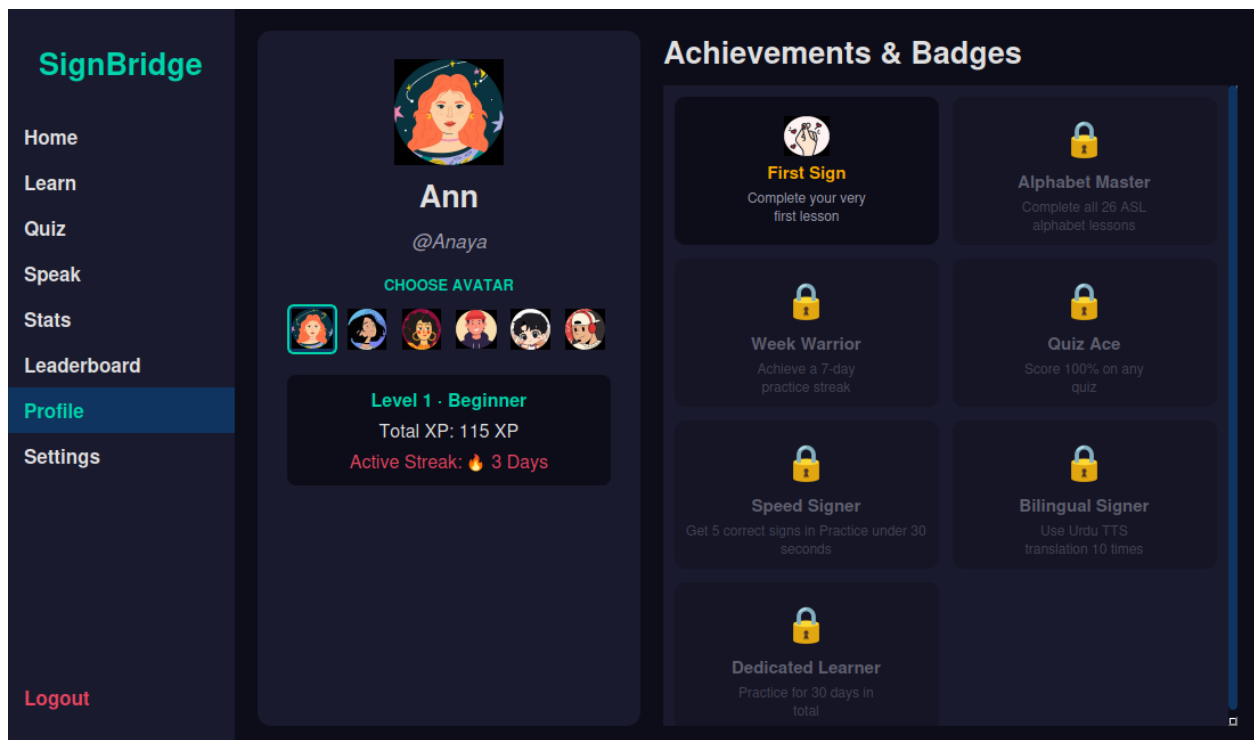
Stats Screen

The Stats screen embeds three Matplotlib figures via FigureCanvasQTAgg inside a QScrollArea. The figures are re-generated on each screen visit from live database data. Charts include a 7-day activity bar chart (signs practiced per day), an accuracy trend line chart (quiz accuracy over sessions), and a curriculum completion donut chart (completed vs remaining lessons per unit).



Profile Screen

The Profile screen shows an avatar image (QLabel with circular crop via QPixmap.scaled()), the display name, username, level badge, total XP, and current streak. Below is an avatar selector grid (QGridLayout of clickable QPushButton widgets) and an earned badges section with PNG badge icons.



6. Stats & Quiz Modules

6.1 Stats Screen — Data & Charts

Summary Tiles

Five metric tiles at the top of the Stats screen provide at-a-glance performance data:

- Total XP — cumulative experience points earned across all activities.
- Streak — current consecutive daily login streak with fire emoji indicator.
- Signs Learned — count of completed lessons versus total available.
- Quizzes Taken — total number of quiz sessions completed.
- Best Accuracy — maximum accuracy achieved in either practice or quiz mode.

Chart 1: 7-Day Activity Bar Chart

Queries the practice and quiz history tables for the past seven days and groups results by date. Renders a Matplotlib bar chart with TEAL (#00d4aa) bars on a dark (#0d0d1a) background, showing daily sign interaction counts. Helps learners identify consistency gaps.

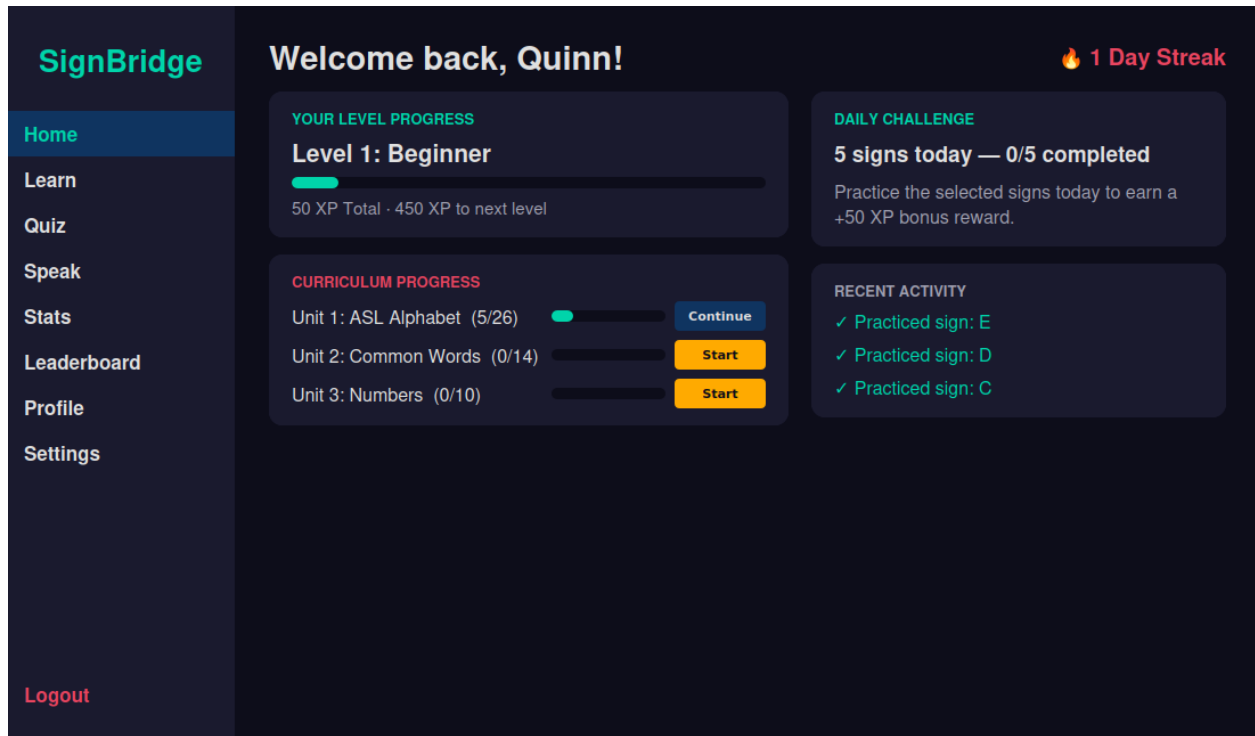
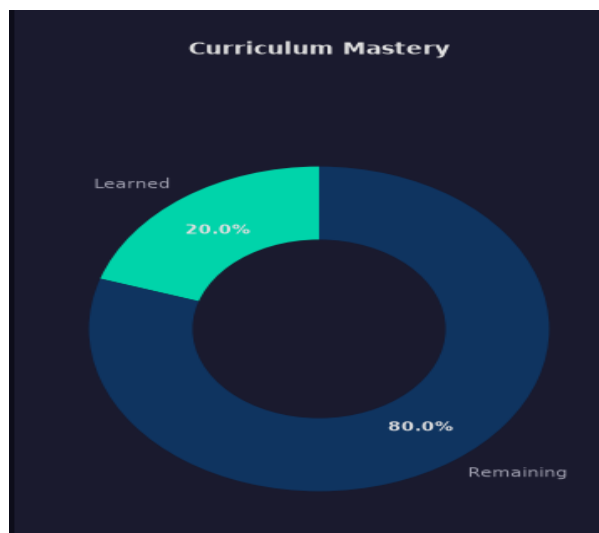


Chart 2: Quiz Accuracy Trend

Pulls the accuracy score from every quiz_results record for the current user, sorted by date. Displays a line chart with circular markers. A flat or rising trend confirms skill consolidation; a dip signals need for targeted review. X-axis shows quiz session number.

Chart 3: Curriculum Completion Donut



Divides all lessons by unit and computes the completion ratio for each. A donut chart (pie with centre hole) shows green segments for completed lessons and grey segments for remaining, with a percentage label in the centre. This provides a high-level curriculum progress overview.

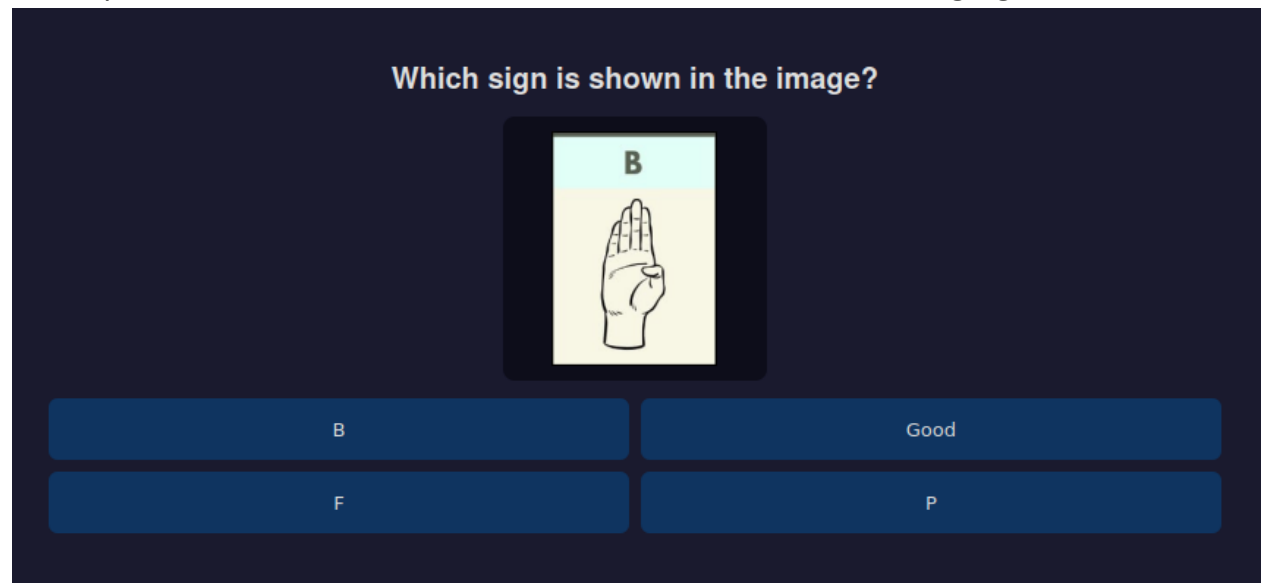
6.2 Quiz Engine — Architecture & Flow

Question Generation

The QuizEngine fetches the learner's completed lesson IDs from the progress table. The SpacedRep module computes a priority weight for each completed sign based on historical accuracy — signs answered incorrectly more often receive higher weights. The engine samples 10 questions using weighted random selection, alternating between FLASHCARD and SIGN_IT types.

FLASHCARD Questions

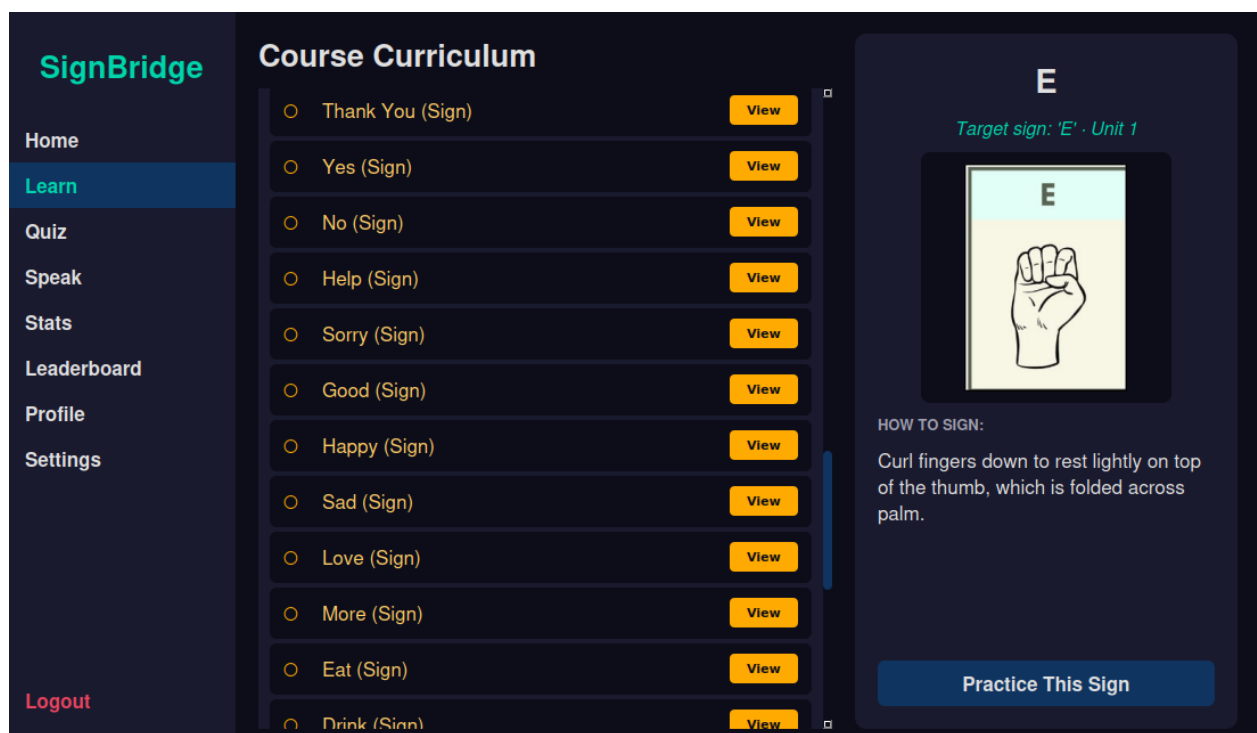
A sign reference image is displayed, and the learner is presented with four multiple-choice letter options. One button click records the answer. Correct answers highlight the button in



teal; incorrect answers reveal the correct option in red while the chosen option turns red. The session auto-advances after 1.5 seconds.

SIGN_IT Questions

The target sign label is shown as text. The learner must physically perform the hand sign in front of the webcam. The live camera feed is shown via QLabel, and a QProgressBar fills as the detector accumulates matched frames. Holding the correct sign for 45 consecutive frames (approximately 1.5 seconds at 30 FPS) counts as a correct answer.



Scoring & Completion

At session end, the QuizScreen displays a results panel showing: score (e.g., 8/10), percentage, XP earned (base 50 XP + 10 per correct, +100 bonus for perfect score), and a per-question breakdown with correct/incorrect indicators. The session is written to quiz_results in SQLite, and badge conditions are re-evaluated (e.g., Quiz Ace badge for 100% accuracy).

Spaced Repetition Update

After each quiz session, the SpacedRep module updates difficulty weights: correct answers decay the weight by a fixed factor; incorrect answers increase it. On the next quiz session, signs with higher difficulty weights appear more frequently in the sampled question set, implementing a simplified Leitner-style review schedule.

7. Future Scope & Real-World Applications

7.1 Technical Improvements

- **Theme:** Light Theme via QSS: Implement a complete QSS stylesheet toggle for a white/light theme. PyQt5 supports runtime style sheet swapping via QApplication.setStyleSheet(), making this straightforward to add.
- **AI:** Model Accuracy: Retrain the TensorFlow classifier on a larger, more diverse dataset (multiple hand sizes, skin tones, lighting conditions) to reduce false positives in real-world environments.
- **Distribution:** Executable Packaging: Package SignBridge as a standalone .exe (Windows via PyInstaller) or .app (macOS) so users do not need Python installed. This is the single biggest barrier to non-developer adoption.
- **TTS:** Offline TTS: Pre-generate all sign audio files and bundle them with the app, eliminating the internet dependency for gTTS. pyttsx3 quality is lower; a bundled gTTS cache is the better solution.
- **Performance:** GPU Inference: Add TensorFlow GPU support detection and enable it when available, reducing inference latency from ~50ms to under 10ms for smoother real-time recognition.
- **Quiz:** More Quiz Types: Add MULTIPLE_CHOICE_IMAGE (pick the image matching a label) and ORDERING (arrange signed letters to spell a word) question types to increase variety.

7.2 Curriculum Expansion Common Phrases Unit: Add a Unit 4 covering multi-sign phrases such as 'How are you?', 'My name is...', 'Nice to meet you', expanding practical communicative utility.

- Numbers Completion: Seed digits 6–9 and extend to double-digit numbers (10, 11, 12...) with new reference image assets.
- Regional Sign Variants: ASL signs have regional and generational variants. A future version could note variant forms and provide multiple reference images per sign.
- Progress Certificates: Generate a downloadable PDF certificate upon curriculum unit completion, useful for learners who wish to demonstrate progress.

7.3 Platform Expansion

- **Web:** Web Application: Port the UI layer to a React frontend with a FastAPI backend, making SignBridge accessible from any browser without installation.
- **Mobile:** Mobile App: Implement a Flutter or React Native version using the device

camera for hand sign recognition, expanding reach to the largest user demographic.

- **Localisation:** Multi-Language TTS: Extend the language system to support additional South Asian languages — Punjabi, Sindhi, Pashto — serving a wider Pakistani audience.
- **Education:** Classroom Mode: Add a teacher dashboard that tracks a class of student accounts, views aggregate progress, assigns lessons, and exports attendance reports.

7.4 Real-World Applications

Deaf & Hard-of-Hearing Communication

The most direct application is training hearing individuals — family members, coworkers, service staff — to communicate with Deaf or hard-of-hearing people in their communities. A mobile version could serve as a real-time ASL-to-text translator in conversations.

School & University Education

SignBridge can be deployed in Pakistani schools as a digital literacy and inclusion tool. Special education departments can use it to train teachers and support staff in basic ASL, reducing communication barriers for hearing-impaired students.

Healthcare Settings

Nurses, doctors, and emergency responders who interact with Deaf patients could use SignBridge for rapid skill-building. The gamified format makes short daily practice sessions sustainable even for professionals with limited time.

Corporate Inclusion Programs

Companies with diversity and inclusion initiatives can deploy SignBridge as part of employee onboarding or HR training to create a more welcoming workplace for Deaf colleagues and clients.

Self-Learning & Accessibility Advocacy

The open-ended self-paced format is ideal for individual learners motivated by personal connection or advocacy goals. The Urdu bilingual layer makes it uniquely suited to the Pakistani context, where no comparable localised tool currently exists.

8. Summary

SignBridge is a complete, functional, and professionally structured desktop application that demonstrates the integration of computer vision, machine learning, database management, gamification, and bilingual natural language processing within a single Python project.

The migration from CustomTkinter to PyQt5 marks a significant maturation of the codebase: the application now uses a signal-slot event model, widget-level QSS styling, embedded Matplotlib charts via the Qt5Agg backend, and a clean QStackedWidget routing system. All non-UI logic (ASL recognition, quiz engine, XP system, TTS, database) was preserved unchanged through the conversion.

The project is deployable today — all 26 ASL alphabet sign images are bundled, five badge icons and six avatar images are included, and the database seeds itself on first launch. The only optional component is the TensorFlow model file; without it, the app runs fully in simulation mode, making it testable on any machine.

In a broader context, SignBridge demonstrates that barrier-free, AI-assisted communication tools can be built at the student project level with modern Python libraries — and that with packaging and community distribution, such tools have genuine real-world impact for Deaf and hearing-impaired communities in Pakistan and beyond.